

Tins: Threshold Computation in the Head Signatures from Normalized Subfield Bilinear Collisions

Algorithm Specifications and Supporting Documentation

Xianfang Wang¹, Renzhang Liu², Shimin Li³, Binbin Tu⁴

¹CETC Cyberspace Security Technology Company Ltd.

² ShengAnXin Technology CO., LTD

³The Third Research Institute of the Ministry of Public Security

⁴School of Cyber Science and Technology, Shandong University

Contact: pqctins@163.com

Contents

1	Introduction	3
2	Mathematical Background and Notations	3
2.1	Notations	3
2.2	Normalized Subfield Bilinear Collision Problem	3
2.3	All-but-one Vector Commitment(AVC)	5
3	High-Level Description	5
3.1	NSBC-in-the-Head	6
3.2	Overview of Tins	7
4	Detailed algorithmic description	10
4.1	Subroutines	10
4.1.1	Finite field and functions	10
4.1.2	Randomness generation and sampling	10
4.1.3	Batched all-but-one vector commitments	11
4.1.4	Commitment to polynomials	12
4.1.5	Computation of polynomial	13
4.2	Key Generation	16
4.3	Signing	16
4.4	Verification	16
5	Parameter Sets	17
6	Security and Known Attacks	17
6.1	Security Proof	17
6.2	Known Attacks	20
7	Performance	21
7.1	Implementation	21
7.2	Known Answer Test Values	22
8	Advantages and Limitations	22
8.1	Advantages	22
8.2	Limitations	22

1 Introduction

In 2007, Ishai et al.[13] proposed an approach to designing Zero-Knowledge Proofs using Multi-Party Computation (MPC). This approach has been widely adopted in recent years for designing post-quantum digital signatures. Examples include the Mirath scheme (based on the Minrank problem)[1], SDitH (based on the syndrome decoding problem)[2], the RYDE scheme (based on Rank Syndrome Decoding)[3], and the FAEST scheme (based on symmetric primitives)[4], among others. MPCitH signatures can provide a good balance between verification time and signature size.

Tins is a post-quantum signature algorithm based on the hardness of the Normalized Subfield Bilinear Collision (NSBC) problem. The NSBC problem was proposed by Huth and Joux[11] in 2024, who also designed an MPCitH-based signature scheme and further introduced a VOLE-in-the-head digital signature based on the same problem in 2025[12]. The **Tins** scheme is designed using the TCitH technique, offering a smaller signature size compared to [11] and a more concise and intuitive signing process compared to [12]. Similar to other approaches, **Tins** originates from the Zero-Knowledge Proof (ZKP) method. It employs the TCitH framework to construct a ZKP protocol for the NSBC problem, which is then transformed into a digital signature using the Fiat-Shamir transform[10].

This document is organized as follows. Section 2 introduces the mathematical background and the NSBC problem. A high-level overview of the **Tins** scheme is presented in Section 3, followed by a detailed specification of its algorithms in Section 4. Section 5 details the parameter sets and their corresponding public-key and signature sizes. We then analyze the security of **Tins** and discuss known attacks in Section 6, and present a performance evaluation in Section 7. Finally, Section 8 concludes with a summary of the scheme's main advantages and limitations.

2 Mathematical Background and Notations

2.1 Notations

Let $\mathbb{F}_q$ denote the finite field with q elements, and let $\mathbb{F}_{q^k}$ be the extension field of degree k over the finite field $\mathbb{F}_q$. Vectors and matrices are respectively denoted by lowercase and uppercase bold letters.

We let $y \leftarrow \mathbf{A}(x)$ denote the operation that runs algorithm **A** on input x and assigns the output to the variable y . The main mathematical notations and variable names employed throughout this paper are summarized in Table 1.

2.2 Normalized Subfield Bilinear Collision Problem

The Normalized Subfield Bilinear Collision (NSBC) problem was proposed in [11], is the underlying hard problem of **Tins**. We give a formal definition in the following.

Definition 1. (Normalized Subfield Bilinear Collision, NSBC) Let $\hat{\mathbf{u}} = (\mathbf{u}, u_{n-2}, u_{n-1}) \in \mathbb{F}_{q^k}^n$, $\hat{\mathbf{v}} = (\mathbf{v}, v_{n-2}, v_{n-1}) \in \mathbb{F}_{q^k}^n$, which are linearly independent over $\mathbb{F}_q$. The Normalized Subfield Bilinear Collision Problem asks, given $\hat{\mathbf{u}}, \hat{\mathbf{v}}$, to find two vectors $\boldsymbol{\alpha}, \boldsymbol{\beta} \in \mathbb{F}_q^{n-2}$ such that $(\langle \mathbf{u}, \boldsymbol{\alpha} \rangle + u_{n-2})(\langle \mathbf{v}, \boldsymbol{\beta} \rangle + v_{n-1}) = (\langle \mathbf{u}, \boldsymbol{\beta} \rangle + u_{n-1})(\langle \mathbf{v}, \boldsymbol{\alpha} \rangle + v_{n-2})$.

Usually, the instances of the NSBC is generated by uniformly sampling the vectors $\boldsymbol{\alpha} = (\alpha_0, \alpha_1, \dots, \alpha_{n-3}) \in \mathbb{F}_q^{n-2}$, $\boldsymbol{\beta} = (\beta_0, \beta_1, \dots, \beta_{n-3}) \in \mathbb{F}_q^{n-2}$. Also, randomly choose $\mathbf{u} = (u_0, u_1, \dots, u_{n-3}) \in \mathbb{F}_{q^k}^{n-2}$, $\mathbf{v} = (v_0, v_1, \dots, v_{n-3}) \in \mathbb{F}_{q^k}^{n-2}$, $u_{n-2} \in \mathbb{F}_{q^k}$, $u_{n-1} \in \mathbb{F}_{q^k}$, $v_{n-2} \in \mathbb{F}_{q^k}$. Lastly compute v_{n-1} as

$$v_{n-1} = \frac{(\sum_{i=0}^{n-3} u_i \beta_i + u_{n-1})(\sum_{i=0}^{n-3} v_i \alpha_i + v_{n-2}) - (\sum_{i=0}^{n-3} u_i \alpha_i + u_{n-2})(\sum_{i=0}^{n-3} v_i \beta_i)}{(\sum_{i=0}^{n-3} u_i \alpha_i + u_{n-2})}.$$

If the denominator $(\sum_{i=0}^{n-3} u_i \alpha_i + u_{n-2})$ happens to be zero, we can choose the new random vectors and start the computation again. If the unlikely case that $\mathbf{u}, \mathbf{v}$, are linearly dependent over $\mathbb{F}_q$ appears, we select new vectors instead.

Table 1: Mathematical and algorithmic notation

Symbol	Meaning
$\{0, 1\}^*$	Set of binary strings of finite length.
$\{0, 1\}^l$	Set of binary strings of length l .
$str_1 str_2$	Concatenation of the strings str_1 and str_2 .
$\mathbb{F}_q$	A finite field of q elements.
$\mathbb{F}_q^n$	The vector space of n -tuples over $\mathbb{F}_q$.
$\mathbb{F}_{q^k}$	A finite field of q^k elements.
$\mathbb{F}_{q^k}^n$	The vector space of n -tuples over $\mathbb{F}_{q^k}$.
$\log$	The base-2 logarithm.
$\otimes$	Bitwise multiplication (AND).
$\oplus$	Bitwise addition (XOR).
$\hat{\mathbf{u}}$	Vector of the public key, $\hat{\mathbf{u}} = (\mathbf{u}, u_{n-2}, u_{n-1}) \in \mathbb{F}_{q^k}^n$.
$\hat{\mathbf{v}}$	Vector of the public key, $\hat{\mathbf{v}} = (\mathbf{v}, v_{n-2}, v_{n-1}) \in \mathbb{F}_{q^k}^n$.
$\boldsymbol{\alpha}$	Vector of the secret key, $\boldsymbol{\alpha} \in \mathbb{F}_q^{n-2}$.
$\boldsymbol{\beta}$	Vector of the secret key, $\boldsymbol{\beta} \in \mathbb{F}_q^{n-2}$.
$\langle \mathbf{a}, \mathbf{b} \rangle$	The inner product of vectors $\mathbf{a}$ and $\mathbf{b}$.
$\boldsymbol{\alpha}_{base}$	Random vector, $\boldsymbol{\alpha}_{base} \in \mathbb{F}_{q^\mu}^{n-2}$.
$\boldsymbol{\beta}_{base}$	Random vector, $\boldsymbol{\beta}_{base} \in \mathbb{F}_{q^\mu}^{n-2}$.
δ	Random element, $\delta \in \mathbb{F}_{q^\mu}$.
δ_{base}	Random element, $\delta_{base} \in \mathbb{F}_{q^\mu}$.
$\phi(i^*)$	Evaluation points, $\phi(i^*) \in \mathbb{F}_{q^\mu}$.
$seed_{pk}$	Seed for the generation of the public key, $seed_{pk} \in \{0, 1\}^\lambda$.
$seed_{sk}$	Seed for the generation of the secret key, $seed_{sk} \in \{0, 1\}^\lambda$.
$tree$	Binary tree of seeds, $tree \in \{0, 1\}^{2\lambda\tau N-1}$.
$rseed$	Root seed of $tree$, $rseed \in \{0, 1\}^\lambda$.
$seeds$	Leaf seeds of $tree$, $seeds \in \{0, 1\}^{\lambda\tau N}$.
$salt$	Salt for domain separation between signatures, $salt \in \{0, 1\}^{2\lambda}$.
msg	Message to be signed, $msg \in \{0, 1\}^*$.
$commit$	Commitments of $seeds$, $commit \in \{0, 1\}^{2\lambda\tau N}$.
$openkey$	$tree commit$, $openkey \in \{0, 1\}^{4\lambda\tau N-1}$.
$path$	Opening of the BAVC leaves in $tree$. $path \in \{0, 1\}^{\lambda \cdot T_{open}}$.
$commit_{i^*}$	Commitments of the unopened BAVC leaves in $tree$, $commit_{i^*} \in \{0, 1\}^{2\lambda\tau}$.
π_{BAVC}	$path commit_{i^*}$, $\pi_{BAVC} \in \{0, 1\}^{\lambda \cdot T_{open} + 2\lambda\tau}$.
h_{com}	Hash digest of $commit$, $h_{com} \in \{0, 1\}^{2\lambda}$.
h_{sh}	Commitment of the witness polynomials, $h_{sh} \in \{0, 1\}^{2\lambda}$.
h_{piop}	Polynomial proof commitment.
ctr	Counter for grinding, $ctr \in \{0, 1\}^{64}$.
σ	Signature output by the signing algorithm, $\sigma \in \{0, 1\}^*$.

2.3 All-but-one Vector Commitment(AVC)

Similar to most Threshold-Computation-in-the-Head (TCiTH)-based signature schemes, we require *all-but-one vector commitment (AVC)* as a foundational tool. We give the definition of an AVC scheme in the following.

Definition 2. (All-but-one Vector Commitment, AVC) An all-but-one vector commitment with message space $\{0, 1\}^\lambda$ and parameter N is defined by the following algorithms:

- Commit() $\rightarrow (h_{com}, \mathbf{key}, (seed_1, \dots, seed_N))$: Output a commitment h_{com} with opening key $\mathbf{key}$ for messages $(seed_1, \dots, seed_N) \in (\{0, 1\}^\lambda)^N$.
- Open($\mathbf{key}, i^*$): On a opening key $\mathbf{key}$ and an index $i^* \in \{1, \dots, N\}$, output an opening π_{AVC} for $\{seed_i\}_{i \neq i^*}$.
- Reconstruct(π_{AVC}, i^*) $\rightarrow (\{seed_i\}_{i \neq i^*}, h_{com})$: On the opening π_{AVC} and an index $i^* \in \{1, \dots, N\}$, output the messages $\{seed_i\}_{i \neq i^*}$, together the commitment h_{com} .

The scheme is said complete if

$$(h_{com}, \mathbf{key}, (seed_1, \dots, seed_N)) \leftarrow \text{Commit}(),$$

$$\pi_{AVC} \leftarrow \text{Open}(\mathbf{key}, i^*)$$

then we have $h'_{com} = h_{com}$ and $seed'_i = seed_i$ for all $i \neq i^*$, where

$$(\{seed'_i\}_{i \neq i^*}, h'_{com}) \leftarrow \text{Reconstruct}(\pi_{AVC}, i^*).$$

AVC also possesses *binding* and *hiding* properties, similar to traditional commitment schemes. *Binding* refers to the cryptographic guarantee that a committed message cannot be altered after commitment, ensuring the committer cannot falsely open the commitment with a different message. *Hiding* ensures the committed message remains confidential to the receiver until explicitly revealed, preventing information leakage during the commitment phase.

In digital signatures, multiple **AVCs** are typically used in parallel. We employ the **BAVC** (Batched All-but-one Vector Commitment, for details, see reference [5].) technology to handle multiple **AVCs**. **BAVC** scheme has the same properties as those in **AVC**. We define **BAVC** as follows.

Definition 3. (Batched All-but-one Vector Commitment, BAVC) A batched all-but-one vector commitment(BAVC) with message space $\{0, 1\}^\lambda$ and parameter (N, τ) is defined by the following algorithms:

- Commit() $\rightarrow \left(h_{com}, \mathbf{key}, \left\{ \left(seed_1^{(e)}, \dots, seed_N^{(e)} \right) \right\}_{e \in [1, \tau]} \right)$: Output a commitment h_{com} with opening key $\mathbf{key}$ for messages $(seed_1^{(e)}, \dots, seed_N^{(e)}) \in (\{0, 1\}^\lambda)^N$ for all $e \in [1, \tau]$.
- Open($\mathbf{key}, \{i^{*(e)}\}_e$): On a opening key $\mathbf{key}$ and τ index $i^{*(e)} \in \{1, \dots, N\}$, output an opening π_{BAVC} for $\{seed_i^{(e)}\}_{e, i \neq i^{*(e)}}$.
- Reconstruct($\pi_{BAVC}, \{i^{*(e)}\}_e$) $\rightarrow (\{seed_i^{(e)}\}_{e, i \neq i^{*(e)}}, h_{com})$: On the opening π_{BAVC} and an index $i^{*(e)} \in \{1, \dots, N\}$, output the messages $\{seed_i^{(e)}\}_{e, i \neq i^{*(e)}}$, together the commitment h_{com} .

3 High-Level Description

Tins relies on the TCiTH framework[9, 8] to build a 3-round zero-knowledge proof of knowledge for the NSBC problem, which is then transformed into a signature scheme using the Fiat-Shamir transform. We present this proof system using the formalism of the Polynomial Interactive Oracle Proofs (PIOP), as presented in [7]. In PIOP, a prover convince a verifier that the prover knows a witness $w = (w_1, w_2, \dots, w_n) \in \mathbb{F}_q^n$ that satisfies some public polynomial relations:

$$f_j(w_1, w_2, \dots, w_n) = 0, 1 \leq j \leq m,$$

where $f_1, \dots, f_m$ are polynomials over $\mathbb{F}_q$ of total degree at most d .

The design philosophy of **Tins** scheme is as follows: We first express a NSBC instance as a system of degree- d constraints. Then we employ the PIOP proof framework to demonstrate that the prover possesses a valid witness, and finally transform this proof into a non-interactive format using the Fiat-Shamir heuristic[10].

3.1 NSBC-in-the-Head

In an NSBC instance, given $\hat{\mathbf{u}} = (\mathbf{u}, u_{n-2}, u_{n-1}) \in \mathbb{F}_{q^k}^n$, $\hat{\mathbf{v}} = (\mathbf{v}, v_{n-2}, v_{n-1}) \in \mathbb{F}_{q^k}^n$, to find two vectors $\alpha, \beta \in \mathbb{F}_q^{n-2}$ such that $(\langle \mathbf{u}, \alpha \rangle + u_{n-2})(\langle \mathbf{v}, \beta \rangle + v_{n-1}) = (\langle \mathbf{u}, \beta \rangle + u_{n-1})(\langle \mathbf{v}, \alpha \rangle + v_{n-2})$. Therefore, we use the proof system to prove that a prover knows the witness $\alpha, \beta \in \mathbb{F}_q^{n-2}$ which satisfy the following constraints with $d = 2$:

$$(\langle \mathbf{u}, \alpha \rangle + u_{n-2})(\langle \mathbf{v}, \beta \rangle + v_{n-1}) - (\langle \mathbf{u}, \beta \rangle + u_{n-1})(\langle \mathbf{v}, \alpha \rangle + v_{n-2}) = 0.$$

Now, we describe the proof system for NSBC instances:

1. The prover uniformly generates degree-1 polynomial $P_\alpha(X) = \alpha \cdot X + \alpha_{base}$, $P_\beta(X) = \beta \cdot X + \beta_{base}$ and $P_\delta(X) = \delta \cdot X + \delta_{base}$ at random, where $\alpha_{base} \in_R \mathbb{F}_{q^\mu}^{n-2}$, $\beta_{base} \in_R \mathbb{F}_{q^\mu}^{n-2}$, $\delta \in_R \mathbb{F}_{q^\mu}$, $\delta_{base} \in_R \mathbb{F}_{q^\mu}$, $\mu | k$. He commits these polynomials using seed trees as in the TCitH framework. The prover reveals the degree-1 polynomials $P(X) \in \mathbb{F}_{q^k}[X]$:

$$P(X) = P_\delta(X) + [(\langle \mathbf{u}, P_\alpha(X) \rangle + u_{n-2} \cdot X)(\langle \mathbf{v}, P_\beta(X) \rangle + v_{n-1} \cdot X) - (\langle \mathbf{u}, P_\beta(X) \rangle + u_{n-1} \cdot X)(\langle \mathbf{v}, P_\alpha(X) \rangle + v_{n-2} \cdot X)]$$

actually, $P(X) = p_{mid} \cdot X + p_{base}$, where $p_{mid} = [(\langle \mathbf{u}, \alpha \rangle + u_{n-2})(\langle \mathbf{v}, \beta_{base} \rangle + v_{n-1}) - (\langle \mathbf{u}, \beta \rangle + u_{n-1})(\langle \mathbf{v}, \alpha_{base} \rangle + v_{n-2})] + \delta$, $p_{base} = [(\langle \mathbf{u}, \alpha_{base} \rangle + u_{n-2})(\langle \mathbf{v}, \beta_{base} \rangle + v_{n-1}) - (\langle \mathbf{u}, \beta_{base} \rangle + u_{n-1})(\langle \mathbf{v}, \alpha_{base} \rangle + v_{n-2})] + \delta_{base}$.

2. The verifier samples a random evaluation point $r \in S$, where $S \subseteq \mathbb{F}_{q^\mu}$, $|S| = N$, and sends it to the prover.
3. The prover reveals the evaluations $\alpha_{eval} = P_\alpha(r)$, $\beta_{eval} = P_\beta(r)$, $\delta_{eval} = P_\delta(r)$ together with a proof π that the evaluations are consistent with the commitment.
4. The verifier checks that the revealed evaluations are consistent with the commitment using π and checks that we have

$$P(r) \stackrel{?}{=} \delta_{eval} + [(\langle \mathbf{u}, \alpha_{eval} \rangle + u_{n-2} \cdot r)(\langle \mathbf{v}, \beta_{eval} \rangle + v_{n-1} \cdot r) - (\langle \mathbf{u}, \beta_{eval} \rangle + u_{n-1} \cdot r)(\langle \mathbf{v}, \alpha_{eval} \rangle + v_{n-2} \cdot r)].$$

Security Analysis. The degree-2 monomial in the right term of $P(X)$ in step 2 is

$$[(\langle \mathbf{u}, \alpha \rangle + u_{n-2})(\langle \mathbf{v}, \beta \rangle + v_{n-1}) - (\langle \mathbf{u}, \beta \rangle + u_{n-1})(\langle \mathbf{v}, \alpha \rangle + v_{n-2})],$$

so the definition of the degree-1 polynomial $P(X)$ is well-defined when the witness is valid. We give the soundness error of the above proof system in the following.

Assume the prover is malicious, that is, the prover don't has a valid witness. Then, the probability of the $P(X)$ is a degree-1 polynomial is 0. That is, $\deg(P(X)) < 2$ if and only if the prover has a valid witness. If the $P(X)$ is not well-defined, the probability that the check in step 4 passes is at most $2/N$, since the degree-2 polynomial $P(X) - P_\delta(X) - (\langle \mathbf{u}, P_\alpha(X) \rangle + u_{n-2} \cdot X)(\langle \mathbf{v}, P_\beta(X) \rangle + v_{n-1} \cdot X) + (\langle \mathbf{u}, P_\beta(X) \rangle + u_{n-1} \cdot X)(\langle \mathbf{v}, P_\alpha(X) \rangle + v_{n-2} \cdot X)$ would have at most 2 roots. So the soundness error of the proof system for NSBC instances is $2/N$. In **Tins**, we need repeat the above proof system τ times in parallel, in order to achieve the desired security. To have a λ -bit security, the parameter τ should satisfy $(\frac{2}{N})^\tau \leq \frac{1}{2^\lambda}$.

Witness polynomials commitment. In the proof system for NSBC instances, we need to commit $P_\alpha(X)$, $P_\beta(X)$ and $P_\delta(X)$. We describe the TCitH method that we can commit random polynomials using seed trees.

- (1). We use an all-but-one vector commitment to sample and commit N seeds $seed_1, \dots, seed_N$.

(2). One expands each $seed_i$ as

$$(\alpha_{rnd,i}, \beta_{rnd,i}, \delta_{rnd,i}) := PRG(seed_i), 1 \leq i \leq N,$$

where PRG is a pseudorandom generator, $\alpha_{rnd,i} \in \mathbb{F}_q^{n-2}$, $\beta_{rnd,i} \in \mathbb{F}_q^{n-2}$, $\delta_{rnd,i} \in \mathbb{F}_{q^\mu}$.

(3). One computes

$$\begin{aligned} (\alpha_{acc}, \beta_{acc}, \delta) &\leftarrow \left(\sum_{i=1}^N \alpha_{rnd,i}, \sum_{i=1}^N \beta_{rnd,i}, \sum_{i=1}^N \delta_{rnd,i} \right), \\ (\alpha_{base}, \beta_{base}) &\leftarrow \left(\sum_{i=1}^N \phi(i) \cdot \alpha_{rnd,i}, \sum_{i=1}^N \phi(i) \cdot \beta_{rnd,i} \right), \\ \delta_{base} &\leftarrow - \sum_{i=1}^N \phi(i) \cdot \delta_{rnd,i} \end{aligned}$$

where $\phi : \{1, \dots, N\} \rightarrow \mathbb{F}_{q^\mu}$ is a public one-to-one function, $\alpha_{acc} \in \mathbb{F}_q^{n-2}$, $\beta_{acc} \in \mathbb{F}_q^{n-2}$, $\delta \in \mathbb{F}_{q^\mu}$, $\alpha_{base} \in \mathbb{F}_{q^\mu}^{n-2}$, $\beta_{base} \in \mathbb{F}_{q^\mu}^{n-2}$, $\delta_{base} \in \mathbb{F}_{q^\mu}$.

(4). One computes $\alpha_{aux} = \alpha + \alpha_{acc}$, $\beta_{aux} = \beta + \beta_{acc}$, reveals the auxiliary value $aux = (\alpha_{aux}, \beta_{aux})$.

(5). One defines $P_\alpha(X) = \alpha \cdot X + \alpha_{base}$, $P_\beta(X) = \beta \cdot X + \beta_{base}$ and $P_\delta(X) = \delta \cdot X + \delta_{base}$.

When the verifier sends a challenge value $\phi(i^*)$ to the prover, requesting the evaluation of the commitment polynomial at $\phi(i^*)$, the prover just reveals all the seeds $\{seed_i\}_{1 \leq i \leq N, i \neq i^*}$ except $seed_{i^*}$. Then the verifier could get $\alpha_{eval} = P_\alpha(\phi(i^*))$, $\beta_{eval} = P_\beta(\phi(i^*))$ and $\delta_{eval} = P_\delta(\phi(i^*))$ as

$$\begin{aligned} \alpha_{eval} &= \phi(i^*) \cdot \alpha_{aux} + \sum_{i=1, i \neq i^*}^N (\phi(i) - \phi(i^*)) \cdot \alpha_{rnd,i}, \\ \beta_{eval} &= \phi(i^*) \cdot \beta_{aux} + \sum_{i=1, i \neq i^*}^N (\phi(i) - \phi(i^*)) \cdot \beta_{rnd,i}, \\ \delta_{eval} &= \sum_{i=1, i \neq i^*}^N (\phi(i^*) - \phi(i)) \cdot \delta_{rnd,i}. \end{aligned}$$

In **Tins**, we need perform the polynomials commitment phase τ times in parallel. We use the **BAVC** scheme describe in [5], utilize a pseudorandom function (PRF) to generate child nodes. As established in the literature, a correlated GGM tree offers a significant efficiency gain over its standard counterpart, halving the number of hash function invocations while also paving the way for an optimized version of **BAVC**. While these improvements necessitate the use of CCR hash functions, they provide no substantial performance benefit over the methods in [5] when measured by the frequency of symmetric primitive calls, particularly for 256-bit and 512-bit signature schemes.

3.2 Overview of Tins

The **Tins** scheme is obtained by applying the Fiat-Shamir transformation to the NSBC-in-the-head proof system. As proposed in [5], we also use a proof-of-work to the Fiat-Shamir hash computation of the last challenge, that is the signer samples a ω -bit zero value $v_{grinding}$. Adopting this strategy allows us to take smaller parameters (N, τ) for large tree, the parameters N, τ and ω will be chosen such that $(\frac{2}{N})^\tau \leq \frac{1}{2^{\lambda-\omega}}$ to achieve a λ -bit security [12, 5]. Description of signature generation and verification in **Tins** is as follows.

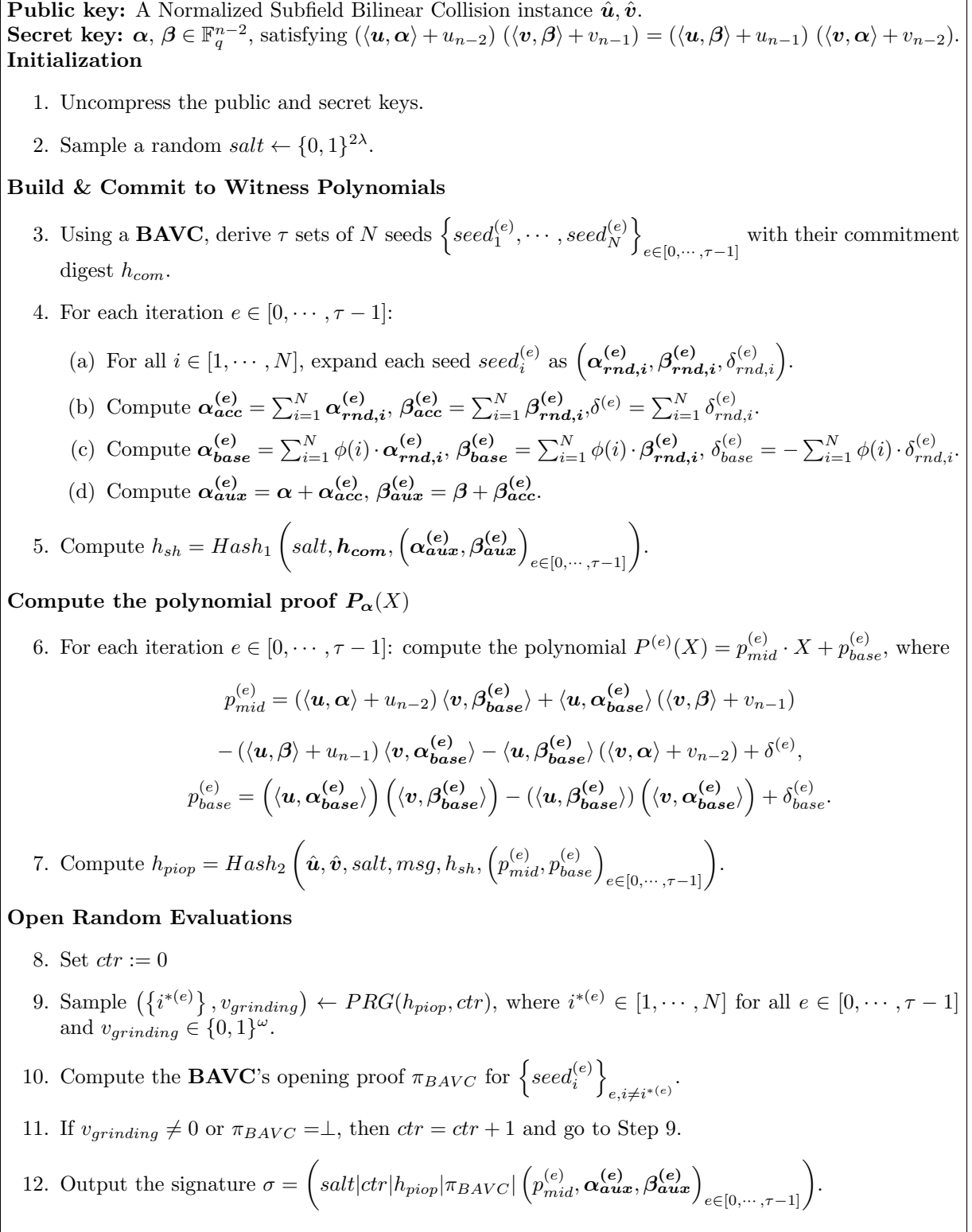


Figure 1: Overview of Tins Sign algorithm

Public key: A Normalized Subfield Bilinear Collision instance $\hat{\mathbf{u}}, \hat{\mathbf{v}}$.

Initialization

1. Uncompress the public keys.
2. Parse the signature as $\left(salt|ctr|h_{piop}|\pi_{BAVC}|\left(p_{mid}^{(e)}, \alpha_{aux}^{(e)}, \beta_{aux}^{(e)}\right)_{e \in [0, \dots, \tau-1]} \right)$.

Computing Opened Evaluations

3. Sample $(\{i^{*(e)}\}, v_{grinding}) \leftarrow PRG(h_{piop}, ctr)$, where $i^{*(e)} \in [1, \dots, N]$ for all $e \in [0, \dots, \tau-1]$ and $v_{grinding} \in \{0, 1\}^\omega$. If $v_{grinding} \neq 0$, abort, output **FLASE**.
4. Using $\pi_{BAVC}, \{i^{*(e)}\}$ and the reconstruction algorithm of the BAVC scheme, recover $\{seed_i^{(e)}\}_{e, i \neq i^{*(e)}}$ and h_{com} .
5. For each iteration $e \in [0, \dots, \tau-1]$:

(a) For all $i \in [1, \dots, N] \setminus \{i^{*(e)}\}$, expand each seed $seed_i^{(e)}$ as $(\alpha_{rnd,i}^{(e)}, \beta_{rnd,i}^{(e)}, \delta_{rnd,i}^{(e)})$.

(b) Compute

$$\begin{aligned} \alpha_{eval}^{(e)} &= \phi(i^{*(e)}) \cdot \alpha_{aux}^{(e)} + \sum_{i=1, i \neq i^{*(e)}}^N (\phi(i) - \phi(i^{*(e)})) \cdot \alpha_{rnd,i}^{(e)}, \\ \beta_{eval}^{(e)} &= \phi(i^{*(e)}) \cdot \beta_{aux}^{(e)} + \sum_{i=1, i \neq i^{*(e)}}^N (\phi(i) - \phi(i^{*(e)})) \cdot \beta_{rnd,i}^{(e)}, \\ \delta_{eval}^{(e)} &= \sum_{i=1, i \neq i^{*(e)}}^N (\phi(i^{*(e)}) - \phi(i)) \cdot \delta_{rnd,i}^{(e)}. \end{aligned}$$

6. Compute $h_{sh} = Hash_1 \left(salt, h_{com}, \left(\alpha_{aux}^{(e)}, \beta_{aux}^{(e)} \right)_{e \in [0, \dots, \tau-1]} \right)$.

Recompute the polynomial proof $P_\alpha(X)$

7. For each iteration $e \in [0, \dots, \tau-1]$:

(a) Compute the evaluation $P_{eval}^{(e)} = P^{(e)}(\phi(i^{*(e)}))$ as

$$\begin{aligned} P_{eval}^{(e)} &= \delta_{eval}^{(e)} + \left(\langle \mathbf{u}, \alpha_{eval}^{(e)} \rangle + u_{n-2} \cdot \phi(i^{*(e)}) \right) \left(\langle \mathbf{v}, \beta_{eval}^{(e)} \rangle + v_{n-1} \cdot \phi(i^{*(e)}) \right) \\ &\quad - \left(\langle \mathbf{u}, \beta_{eval}^{(e)} \rangle + u_{n-1} \cdot \phi(i^{*(e)}) \right) \left(\langle \mathbf{v}, \alpha_{eval}^{(e)} \rangle + v_{n-2} \cdot \phi(i^{*(e)}) \right). \end{aligned}$$

(b) Compute $p_{base}^{(e)} = P_{eval}^{(e)} - p_{mid}^{(e)} \cdot \phi(i^{*(e)})$.

8. Compute $h'_{piop} = Hash_2 \left(\hat{\mathbf{u}}, \hat{\mathbf{v}}, salt, msg, h_{sh}, \left(p_{mid}^{(e)}, p_{base}^{(e)} \right)_{e \in [0, \dots, \tau-1]} \right)$.

Verification

9. Check that $h'_{piop} \stackrel{?}{=} h_{piop}$.

Figure 2: Overview of Tins Verify algorithm

4 Detailed algorithmic description

4.1 Subroutines

4.1.1 Finite field and functions

In **Tins**, we let $q = 2$. For integer a or $a \in \mathbb{F}_{q^k}$, $\text{bin}(a)$ is the binary representation of a . Every element in $\mathbb{F}_{q^k}$ is represented by a degree $k - 1$ polynomial $a_{k-1}x^{k-1} + \dots + a_1x + a_0$, where $a_0, \dots, a_{k-1} \in \mathbb{F}_q$. We use k bits to store a single $\mathbb{F}_{q^k}$ element. The multiplication of two elements in $\mathbb{F}_{q^k}$ is implemented as a polynomial multiplication modulo an irreducible polynomial $f(x)$, the degree of $f(x)$ is k . For an integer b , bitstring s , $\text{LSB}(s, b)$ is the leftmost b bits of s , $\text{RSB}(s, b)$ is the rightmost b bits of s . $\text{InG}(s)$ is the integer representation of s . For a bitstring s , $s \ll u$ denotes a left shift by u bits.

Let $\text{PRG}(\text{seed}, \text{klen})$ is the function that input the bitstring seed and integer klen , output the bitstring of length klen . Let $\text{Hash}_i() : \{0, 1\}^* \rightarrow \{0, 1\}^{2^\lambda}$, $1 \leq i \leq 3$, be secure hash functions with domain separation as follows:

$$\text{Hash}_1(\text{data}) := \text{Hash}(1, \text{data})$$

$$\text{Hash}_2(\text{data}) := \text{Hash}(2, \text{data})$$

$$\text{Hash}_3(\text{data}) := \text{Hash}(3, \text{data})$$

where each of the prefix 1, 2, and 3 is encoded in one byte.

$\text{Encrypt} - \lambda(\text{key}, \text{msg})$ output the encryption $c \in \{0, 1\}^\lambda$ of the block $\text{msg} \in \{0, 1\}^\lambda$ using the key $\text{key} \in \{0, 1\}^\lambda$. The selection of the particular block cipher depends on the security level.

4.1.2 Randomness generation and sampling

$\text{TRG.Seed}(\lambda)$ returns a random binary string of length λ . $\text{PRG}(\text{seed}, \text{klen})$ takes a bit string seed and an integer klen as input, and outputs a pseudo-random bit string of length klen . We treat $\text{TRG.Seed}(\lambda)$ and $\text{PRG}(\text{seed}, \text{klen})$ as black box, and in practical implementation, it can be concretely realized based on specific circumstances.

$\text{SampleFieldElement}(\text{seed}, L, \text{offset})$ takes a seed of length λ , an integer L and an integer offset as input, and outputs L pseudo-random elements in the finite field $\mathbb{F}_q$, this algorithm serves as a foundational component for subsequent vector sampling. The offset in this algorithm is used to mark the starting position of the sampled elements.

Algorithm 1 $\text{SampleFieldElement}(\text{seed}, L, \text{offset})$

Require: : $\text{seed} \in \{0, 1\}^\lambda$, integer L

- 1: $s_{\text{out}} = \text{PRG}(\text{seed}, L + \text{offset})$
 - 2: $s = \text{RSB}(s_{\text{out}}, L)$
 - 3: **for** $0 \leq i \leq L - 1$ **do**
 - 4: $a_i = \text{LSB}(s, 1)$
 - 5: $s \ll 1$
 - 6: **end for**
 - 7: **Output:** $(a_0, a_1, \dots, a_{L-1})$
-

$\text{ExpandPublicVector}(\text{seed}_{pk})$ takes the public key seed seed_{pk} as input, outputs the public key vector $\hat{\mathbf{u}} = (\mathbf{u}, u_{n-2}, u_{n-1}) \in \mathbb{F}_{q^k}^n$, $\mathbf{v} \in \mathbb{F}_{q^k}^{n-2}$ and $v_{n-2} \in \mathbb{F}_{q^k}$.

$\text{ExpandSecretVector}(\text{seed}_{sk}, \text{seed}_{pk})$ takes the public key seed seed_{pk} and the private key seed seed_{sk} as input, outputs the public key vectors $\hat{\mathbf{u}} = (\mathbf{u}, u_{n-2}, u_{n-1}) \in \mathbb{F}_{q^k}^n$, $\hat{\mathbf{v}} = (\mathbf{v}, v_{n-2}, v_{n-1}) \in \mathbb{F}_{q^k}^n$, and the private key vectors $\boldsymbol{\alpha}, \boldsymbol{\beta} \in \mathbb{F}_q^{n-2}$. These algorithms provide a method to derive public and private key vectors from seeds.

Algorithm 2 *ExpandPublicVector*($seed_{pk}$)

Require: : $seed_{pk} \in \{0, 1\}^\lambda$

- 1: $\mathbf{a} = \text{SampleFieldElement}(seed_{pk}, (2n - 1)k, 0)$
 - 2: $(\hat{\mathbf{u}}, \mathbf{v}, v_{n-2}) \leftarrow \mathbf{a}$
 - 3: **Output:** $(\hat{\mathbf{u}}, \mathbf{v}, v_{n-2})$
-

Algorithm 3 *ExpandSecretVector*($seed_{sk}, seed_{pk}$)

Require: : $seed_{sk} \in \{0, 1\}^\lambda, seed_{pk} \in \{0, 1\}^\lambda$

- 1: $(\hat{\mathbf{u}}, \mathbf{v}, v_{n-2}) \leftarrow \text{ExpandPublicVector}(seed_{pk})$
 - 2: $cur_offset = 0$
 - 3: $\mathbf{a} = \text{SampleFieldElement}(seed_{sk}, 2n - 4, cur_offset)$
 - 4: $(\alpha, \beta) \leftarrow \mathbf{a}$
 - 5: **if** $(\sum_{i=0}^{n-3} u_i \alpha_i + u_{n-2} = 0 \pmod{q^k})$ **then**
 - 6: $cur_offset = cur_offset + (2n - 4)$
 - 7: return to step 3
 - 8: **end if**
 - 9: $v_{n-1} = \frac{(\sum_{i=0}^{n-3} u_i \beta_i + u_{n-1})(\sum_{i=0}^{n-3} v_i \alpha_i + v_{n-2}) - (\sum_{i=0}^{n-3} u_i \alpha_i + u_{n-2})(\sum_{i=0}^{n-3} v_i \beta_i)}{(\sum_{i=0}^{n-3} u_i \alpha_i + u_{n-2})}$
 - 10: **if** $(\hat{\mathbf{u}} = \gamma \hat{\mathbf{v}} \text{ for some } \gamma \in \mathbb{F}_q)$ **then**
 - 11: $cur_offset = cur_offset + (2n - 4)$
 - 12: return to step 3
 - 13: **end if**
 - 14: let $\hat{\mathbf{v}} = (\mathbf{v}, v_{n-2}, v_{n-1})$
 - 15: **Output:** $(\hat{\mathbf{u}}, \hat{\mathbf{v}}, \alpha, \beta)$
-

4.1.3 Batched all-but-one vector commitments

ChildrenNodesGen($salt, seed, idx$) provides a method to generate child nodes from a parent node. It takes $salt \in \{0, 1\}^{2\lambda}, seed \in \{0, 1\}^\lambda$, and integer idx as input, and outputs two child node seeds $seed_l, seed_r$. This algorithm serves as a fundamental component for constructing GGM trees.

Algorithm 4 *ChildrenNodesGen*($salt, seed, idx$)

Require: : $salt \in \{0, 1\}^{2\lambda}, seed \in \{0, 1\}^\lambda$, intger idx

- 1: $msg_0 = LSB(salt, \lambda) \oplus (bin_{\lambda-40}(3) || bin_{32}(idx) || bin_8(0))$
 - 2: $msg_1 = LSB(salt, \lambda) \oplus (bin_{\lambda-40}(3) || bin_{32}(idx) || bin_8(1))$
 - 3: $seed_l = \text{Encrypt} - \lambda(seed, msg_0)$
 - 4: $seed_r = \text{Encrypt} - \lambda(seed, msg_1)$
 - 5: **Output:** $(seed_l, seed_r)$
-

BAVC.Commit($salt, rseed$) takes the root node $rseed$ to generate τN leaf nodes and the total commitment value h_{com} of the leaf nodes. It outputs the tree's secret key, consisting of the tree node values $tree[]$ and individual leaf node commitments $commit[][]$. These τN leaf nodes are utilized in τ iterations, with N leaf nodes employed in each iteration to generate polynomial commitments.

BAVC.Open($key, \{i^*[e]\}_{0 \leq e \leq \tau-1}$) takes the tree's secret key key and τ challenge indices $i^*[e]$ as input, and outputs the corresponding proofs for these challenge indices. These proofs consist of the commitments of the leaf nodes associated with the τ challenge indices and the verification paths of other leaf nodes in the tree.

BAVC.Rec($i^*[e]_{0 \leq e \leq \tau-1}, \pi_{BAVC}, salt$) takes τ challenge indices $i^*[e]$, their corresponding proofs π_{BAVC} , and $salt$ as input, and reconstructs all leaf nodes except those at the challenged positions, along with the final commitment value h_{com} for all leaf nodes.

Algorithm 5 *BAVC.Commit*(*salt*, *rseed*)

Require: : $\text{salt} \in \{0, 1\}^{2\lambda}$, $\text{rseed} \in \{0, 1\}^\lambda$

- 1: $\text{tree}[0] = \text{rseed}$
- 2: **for** $0 \leq i \leq \tau N - 2$ **do**
- 3: $(\text{tree}[2i + 1], \text{tree}[2i + 2]) = \text{ChildrenNodesGen}(\text{salt}, \text{tree}[i], i)$
- 4: **end for**
- 5: **for** $0 \leq e \leq \tau - 1$ **do**
- 6: **for** $0 \leq i \leq N - 1$ **do**
- 7: $\text{seeds}[e][i] = \text{tree}[(\tau N - 1) + i\tau + e]$
- 8: $\text{commit}[e][i] = \text{Hash}_3(\text{salt}, \text{seeds}[e][i], (\tau N - 1) + i\tau + e)$
- 9: **end for**
- 10: **end for**
- 11: $h_{\text{com}} = \text{Hash}_3(\{\text{commit}[e][i]\}_{0 \leq e \leq \tau - 1, 0 \leq i \leq N - 1})$
- 12: $\text{key} = \text{tree} || \text{commit}$
- 13: **Output:** ($\text{seeds}, h_{\text{com}}, \text{key}$)

Algorithm 6 *BAVC.Open*(*key*, $\{i^*[e]\}_{0 \leq e \leq \tau - 1}$)

Require: : $\text{key} = \text{tree} || \text{commit}$, $\{i^*[e]\}_{0 \leq e \leq \tau - 1}$, $i^*[e] \in [0, N - 1]$

- 1: $(\text{tree}, \text{commit}) \leftarrow \text{key}$
- 2: $\text{hidden} \leftarrow \{(\tau N - 1) + i^*[e]\tau + e : e \in \{0, \dots, \tau - 1\}\}$
- 3: $\text{revealed} \leftarrow \{\tau N - 1, \tau N, \dots, 2\tau N - 2\} \setminus \text{hidden}$
- 4: **for** (i from $\tau N - 2$ down to 0) **do**
- 5: **if** $(2i + 1) \in \text{revealed}$ and $(2i + 2) \in \text{revealed}$ **then**
- 6: $\text{revealed} \leftarrow (\text{revealed} \setminus \{2i + 1, 2i + 2\}) \cup \{i\}$
- 7: **end if**
- 8: **end for**
- 9: **if** $|\text{revealed}| \geq T_{\text{open}}$ **then**
- 10: return $\perp$
- 11: **end if**
- 12: $\text{path} \leftarrow \emptyset$
- 13: **for** $(0 \leq i \leq 2\tau N - 2)$ **do**
- 14: **if** $i \in \text{revealed}$ **then**
- 15: $\text{path} \leftarrow \text{path} || \text{tree}[i]$
- 16: **end if**
- 17: **end for**
- 18: $\text{commit}_{i^*} \leftarrow \text{commit}[0][i^*[0]] || \dots || \text{commit}[\tau - 1][i^*[\tau - 1]]$
- 19: $\pi_{\text{BAVC}} \leftarrow \text{path} || \text{commit}_{i^*}$
- 20: **Output:** π_{BAVC}

4.1.4 Commitment to polynomials

ExpandSeeds(*salt*, *seed*) takes the seed of the leaf nodes as input and outputs the random values $\alpha_{\text{rnd}} \in \mathbb{F}_q^{n-2}$, $\beta_{\text{rnd}} \in \mathbb{F}_q^{n-2}$, $\delta \in \mathbb{F}_{q^\mu}$ derived from the leaf nodes, which serves as a foundational component for polynomial commitments.

CommitPoly(*salt*, *rseed*, α , β) takes the root node *rseed* and private keys α, β as input. It constructs all leaf nodes of the tree, computes linear polynomial vectors and related public auxiliary values $\text{aux}[]$ using these leaf nodes and private keys, and ultimately calculates the commitment value h_{sh} for these polynomials. The algorithm outputs: the constant terms *base* of the polynomial vectors, the commitment h_{sh} , the tree's secret key *key*, the public auxiliary values $\text{aux}[]$.

Algorithm 7 *BAVC.Rec*($i^*[e]_{0 \leq e \leq \tau-1}, \pi_{BAVC}, salt$)

Require: : $salt \in \{0, 1\}^{2\lambda}$, $i^*[e]_{0 \leq e \leq \tau-1} \in [0, N-1]$, $\pi_{BAVC} = path || commit_{i^*}$

```
1:  $(path, commit_{i^*}) = \pi_{BAVC}$ 
2:  $hidden \leftarrow \{(\tau N - 1) + i^*[e]\tau + e : e \in \{0, \dots, \tau - 1\}\}$ 
3:  $revealed \leftarrow \{\tau N - 1, \tau N, \dots, 2\tau N - 2\} \setminus hidden$ 
4: for ( $i$  from  $\tau N - 2$  down to 0) do
5:   if  $(2i + 1) \in revealed$  and  $(2i + 2) \in revealed$  then
6:      $revealed \leftarrow (revealed \setminus \{2i + 1, 2i + 2\}) \cup \{i\}$ 
7:   end if
8: end for
9:  $tree[0] \leftarrow \emptyset, \dots, tree[2\tau N - 2] \leftarrow \emptyset$ 
10: for ( $0 \leq i \leq 2\tau N - 2$ ) do
11:   if  $i \in revealed$  then
12:      $(tree[i] || path) \leftarrow path$ 
13:   end if
14:   if  $(tree[i] \neq \emptyset$  and  $i \leq \tau N - 1$  then
15:      $(tree[2i + 1], tree[2i + 2]) = \mathbf{ChildrenNodesGen}(salt, tree[i], i)$ 
16:   end if
17: end for
18: for ( $0 \leq e \leq \tau - 1$ ) do
19:   for ( $0 \leq i \leq N - 1$ ) do
20:     if  $i \neq i^*[e]$  then
21:        $seed[e][i] = tree[(\tau N - 1) + i\tau + e]$ 
22:        $com[e][i] = Hash_3(salt, seeds[e][i], (\tau N - 1) + i\tau + e)$ 
23:     else
24:        $seed[e][i] \leftarrow \emptyset$ 
25:        $(com[e][i] || commit_{i^*}) \leftarrow commit_{i^*}$ 
26:     end if
27:   end for
28: end for
29:  $h_{com} = Hash_3(\{com[e][i]\}_{0 \leq e \leq \tau-1, 0 \leq i \leq N-1})$ 
30: Output:  $(h_{com}, seeds)$ 
```

Algorithm 8 *ExpandSeeds*($salt, seed$)

Require: : $salt \in \{0, 1\}^{2\lambda}$, $seed \in 0, 1^{\{\lambda\}}$

```
1:  $\mathbf{a} = \mathbf{SampleFieldElement}(salt || seed, 2n - 4 + \mu, 0)$ 
2:  $(\alpha_{rnd} \in \mathbb{F}_q^{n-2}, \beta_{rnd} \in \mathbb{F}_q^{n-2}, \delta \in \mathbb{F}_{q^\mu}) \leftarrow \mathbf{a}$ 
3: Output:  $(\alpha_{rnd}, \beta_{rnd}, \delta)$ 
```

4.1.5 Computation of polynomial

ComputePoly($base[e], \delta[e], \alpha, \beta, \hat{u}, \hat{v}$) computes the polynomial $P(X) = p_{mid} \cdot X + p_{base}$ using the private key vectors α, β and the random values $\alpha_{base}, \beta_{base}, \delta_{base}$, generated from leaf nodes, ultimately outputting the p_{mid} and p_{base} .

ExpandChallengePoint($seed, ctr$) takes a bitstring $seed$ and an integer ctr as input, generates τ challenge indices $i^*[e]$ and a ω -bit string $v_{grinding}$. These challenge indices are used as inputs to *BAVC.Open* for generating corresponding proofs.

OpenRandomEva(key, h_{piop}) takes the tree's secret key key and the $Hash_2$ output value h_{piop} as input. It first invokes *ExpandChallengePoint*(h_{piop}, ctr) to generate challenge indices $\{i^*[e]\}_{0 \leq e \leq \tau-1}$, then calls *BAVC.Open*($key, \{i^*[e]\}_{0 \leq e \leq \tau-1}$) repeatedly until a valid π_{BAVC} and $v_{grinding}$ is obtained,

Algorithm 9 *CommitPoly*($\text{salt}, rseed, \alpha, \beta$)

Require: : $\text{salt} \in \{0, 1\}^{2\lambda}, rseed \in \{0, 1\}^\lambda, \alpha \in \mathbb{F}_q^{n-2}, \beta \in \mathbb{F}_q^{n-2}$

```
1: ( $\text{seeds}, h_{com}, \text{key}$ ) = BAVC.Commit( $\text{salt}, rseed$ )
2: for  $0 \leq e \leq \tau - 1$  do
3:   ( $\alpha_{acc}, \beta_{acc}, \delta_{acc}$ ) = ( $\mathbf{0}, \mathbf{0}, 0$ )  $\in \mathbb{F}_q^{n-2} \times \mathbb{F}_q^{n-2} \times \mathbb{F}_{q^\mu}$ 
4:   ( $\alpha_{base}, \beta_{base}, \delta_{base}$ ) = ( $\mathbf{0}, \mathbf{0}, 0$ )  $\in \mathbb{F}_{q^\mu}^{n-2} \times \mathbb{F}_{q^\mu}^{n-2} \times \mathbb{F}_{q^\mu}$ 
5:   for  $0 \leq i \leq N - 1$  do
6:     ( $\alpha_{rnd}, \beta_{rnd}, \delta_{rnd}$ ) = ExpandSeeds( $\text{salt}, \text{seeds}[e][i]$ )
7:     ( $\alpha_{acc}, \beta_{acc}, \delta_{acc}$ ) = ( $\alpha_{acc}, \beta_{acc}, \delta_{acc}$ ) + ( $\alpha_{rnd}, \beta_{rnd}, \delta_{rnd}$ )
8:     ( $\alpha_{base}, \beta_{base}, \delta_{base}$ ) = ( $\alpha_{base}, \beta_{base}, \delta_{base}$ ) + ( $\phi(i)\alpha_{rnd}, \phi(i)\beta_{rnd}, -\phi(i)\delta_{rnd}$ )
9:   end for
10:   $\text{aux}[e] = (\alpha + \alpha_{acc}, \beta + \beta_{acc})$ 
11:   $\text{base}[e] = (\alpha_{base}, \beta_{base}, \delta_{base})$ 
12:   $\delta[e] = \delta_{acc}$ 
13: end for
14:  $h_{sh} = \text{Hash}_1(\text{salt}, h_{com}, \text{aux}[0], \dots, \text{aux}[\tau - 1])$ 
15: Output: ( $\text{base}, \delta, h_{sh}, \text{key}, \text{aux}$ )
```

Algorithm 10 *ComputePoly*($\text{base}[e], \delta[e], \alpha, \beta, \hat{u}, \hat{v}$)

Require: : $\text{base}[e] = (\alpha_{base}, \beta_{base}, \delta_{base}), \delta[e] = \delta_{acc}, \alpha \in \mathbb{F}_q^{n-2}, \beta \in \mathbb{F}_q^{n-2}, \hat{u} = (u, u_{n-2}, u_{n-1}) \in \mathbb{F}_{q^k}^n$,

$\hat{v} = (v, v_{n-2}, v_{n-1}) \in \mathbb{F}_{q^k}^n$

```
1: ( $\alpha_{base}, \beta_{base}, \delta_{base}$ ) =  $\text{base}[e]$ 
2:  $p_{mid} = (\langle u, \alpha \rangle + u_{n-2}) \langle v, \beta_{base} \rangle + \langle u, \alpha_{base} \rangle (\langle v, \beta \rangle + v_{n-1}) - (\langle u, \beta \rangle + u_{n-1}) \langle v, \alpha_{base} \rangle -$   

    $\langle u, \beta_{base} \rangle (\langle v, \alpha \rangle + v_{n-2}) + \delta[e]$ 
3:  $p_{base} = (\langle u, \alpha_{base} \rangle) (\langle v, \beta_{base} \rangle) - (\langle u, \beta_{base} \rangle) (\langle v, \alpha_{base} \rangle) + \delta_{base}$ 
4: Output: ( $p_{mid}, p_{base}$ )
```

Algorithm 11 *ExpandChallengePoint*(seed, ctr)

Require: : $\text{seed} \in \{0, 1\}^{2\lambda}$, integer ctr

```
1:  $u = \lceil \log N \rceil$ 
2:  $s_{out} = \text{PRG}(\text{seed} || \text{bin}_{64}(\text{ctr}), \tau u + \omega)$ 
3: for  $0 \leq e \leq \tau - 1$  do
4:    $i^*[e] = \text{InG}(\text{LSB}(s_{out}), u)$ 
5:    $i^*[e] = i^*[e] \bmod N$ 
6:    $s_{out} = s_{out} \ll u$ 
7: end for
8:  $v_{grinding} = s_{out}$ 
9: Output: ( $\{i^*[e]\}_{0 \leq e \leq \tau-1}, v_{grinding}$ )
```

ultimately producing the BAVC proof π_{BAVC} as output.

ComputeEva($\text{salt}, \text{ctr}, h_{piop}, \pi_{BAVC}, \text{aux}$) is invoked during signature verification to reconstruct the challenge values $\{i^*[e]\}_{0 \leq e \leq \tau-1}$ and the corresponding leaf nodes *seeds* in the tree, subsequently evaluating the polynomials $P_\alpha(X) = \alpha \cdot X + \alpha_{base}$, $P_\beta(X) = \beta \cdot X + \beta_{base}$ and $P_\delta(X) = \delta \cdot X + \delta_{base}$. at the challenge point $\phi(i^*[e])$.

RecomputePolyProof($\text{point}, \text{eval}, \hat{u}, \hat{v}, p_{mid}$) takes the polynomial evaluation *eval*, challenge point *point*, public key parameters $\hat{u}, \hat{v}$ and signature component p_{mid} as input, then recomputes the constant term matrix p_{base} of the polynomial $P(X)$. This algorithm serves as a critical component in the signature verification protocol, being invoked during the *Verify*().

Algorithm 12 *OpenRandomEva*(key, h_{piop})

Require: : $key = tree || commit, h_{piop} \in \{0, 1\}^{2\lambda}$

```
1:  $ctr = 0$ 
2: while (1) do
3:    $(\{i^*[e]\}_{0 \leq e \leq \tau-1}, v_{grinding}) = \mathbf{ExpandChallengePoint}(h_{piop}, ctr)$ 
4:   if ( $v_{grinding} == 0$ ) then
5:     if ( $\mathbf{BAVC.Open}(key, \{i^*[e]\}_{0 \leq e \leq \tau-1})$  is true) then
6:       break;
7:     end if
8:   end if
9:    $ctr = ctr + 1$ 
10: end while
11: Output: ( $ctr, \pi_{BAVC}$ )
```

Algorithm 13 *ComputeEva*($salt, ctr, h_{piop}, \pi_{BAVC}, aux$)

Require: : $salt \in \{0, 1\}^{2\lambda}$, integer ctr , $h_{piop} \in \{0, 1\}^{2\lambda}$, $\pi_{BAVC} = path || commit_{i^*}$, $aux, \hat{u}, \hat{v}$

```
1:  $(\{i^*[e]\}_{0 \leq e \leq \tau-1}, v_{grinding}) = \mathbf{ExpandChallengePoint}(h_{piop}, ctr)$ 
2: if  $v_{grinding} \neq 0$  then
3:   abort()
4: end if
5:  $(h_{com}, seeds) = \mathbf{BAVC.Rec}(i^*[e]_{0 \leq e \leq \tau-1}, \pi_{BAVC}, salt)$ 
6:  $h_{sh} = \mathbf{Hash}_1(salt, h_{com}, aux[0], \dots, aux[\tau-1])$ 
7: for  $0 \leq e \leq \tau-1$  do
8:    $(\alpha_{eval}, \beta_{eval}, \delta_{eval}) = (\mathbf{0}, \mathbf{0}, \mathbf{0}) \in \mathbb{F}_{q^\mu}^{n-2} \times \mathbb{F}_{q^\mu}^{n-2} \times \mathbb{F}_{q^\mu}$ 
9:   for  $0 \leq i \leq N-1$  do
10:    if  $i \neq i^*[e]$  then
11:       $(\alpha_{rnd}, \beta_{rnd}, \delta_{rnd}) = \mathbf{ExpandSeeds}(salt, seed[e][i])$ 
12:       $a = \phi(i) - \phi(i^*[e])$ 
13:       $(\alpha_{eval}, \beta_{eval}, \delta_{eval}) = (\alpha_{eval}, \beta_{eval}, \delta_{eval}) + (a \cdot \alpha_{rnd}, a \cdot \beta_{rnd}, -a \cdot \delta_{rnd})$ 
14:    end if
15:  end for
16:   $point[e] = \phi(i^*[e])$ 
17:   $(\alpha_{aux}, \beta_{aux}) = aux[e]$ 
18:   $(\alpha_{eval}, \beta_{eval}) = (\alpha_{eval}, \beta_{eval}) + (point[e] \cdot \alpha_{aux}, point[e] \cdot \beta_{aux})$ 
19:   $evals[e] = (\alpha_{eval}, \beta_{eval}, \delta_{eval})$ 
20: end for
21: Output: ( $eval, point, h_{sh}, v_{grinding}$ )
```

Algorithm 14 *RecomputePolyProof*($point, eval, \hat{u}, \hat{v}, p_{mid}$)

Require: : $point = \phi(i^*[e]) \in \mathbb{F}_{q^\mu}$, $eval = (\alpha_{eval}[e], \beta_{eval}[e], \delta_{eval}[e]), \hat{u}, \hat{v}, p_{mid}$

```
1:  $(\alpha_{eval}[e], \beta_{eval}[e], \delta_{eval}[e]) = eval$ 
2:  $P_{eval} = \delta_{eval}[e] + (\langle \mathbf{u}, \alpha_{eval}[e] \rangle + u_{n-2} \cdot \phi(i^*[e])) (\langle \mathbf{v}, \beta_{eval}[e] \rangle + v_{n-1} \cdot \phi(i^*[e])) -$   

 $(\langle \mathbf{u}, \beta_{eval}[e] \rangle + u_{n-1} \cdot \phi(i^*[e])) (\langle \mathbf{v}, \alpha_{eval}[e] \rangle + v_{n-2} \cdot \phi(i^*[e]))$ 
3:  $p_{base} = P_{eval} - p_{mid} \cdot point$ 
4: Output:  $p_{base}$ 
```

4.2 Key Generation

Tins' key generation algorithm $\text{KeyGen}(\lambda, n, k, q)$ takes system parameters as input and outputs a public key $\mathbf{pk}$ and a secret key $\mathbf{sk}$. The public key $\mathbf{pk}$ comprises a public key seed seed_{pk} and a value v_{n-1} , while the secret key $\mathbf{sk}$ contains both the public key seed seed_{pk} and a private key seed seed_{sk} .

Algorithm 15 $\text{KeyGen}(\lambda, n, k, q)$

```

1:  $\text{seed}_{sk} \leftarrow \text{TRG.Seed}(\lambda)$ 
2:  $\text{seed}_{pk} \leftarrow \text{TRG.Seed}(\lambda)$ 
3:  $(\hat{\mathbf{u}}, \hat{\mathbf{v}}, \alpha, \beta) = \text{ExpandSecretVector}(\text{seed}_{sk}, \text{seed}_{pk})$ 
4:  $\mathbf{pk} \leftarrow (\text{seed}_{pk}, v_{n-1})$ 
5:  $\mathbf{sk} \leftarrow (\text{seed}_{sk}, \text{seed}_{pk})$ 
6: Output:  $\mathbf{pk}, \mathbf{sk}$ 

```

4.3 Signing

Tins' digital signature sign algorithm $\text{Sign}(\mathbf{sk}, \text{msg})$ takes a message msg , secret key $\mathbf{sk}$ as inputs. The algorithm proceeds as follows: derive public-private key vectors $(\hat{\mathbf{u}}, \hat{\mathbf{v}})$ using $\text{ExpandSecretVector}(\text{seed}_{sk}, \text{seed}_{pk})$; compute commitment values for secret polynomials; generate τ linear polynomial coefficients $(p_{mid}[e], p_{base}[e])$, and then compute proof π_{BAC} . Output the composite signature $\sigma = (\text{salt}, \text{ctr}, h_{piop}, \pi_{BAC}, p_{mid}, \text{aux})$. The maximum bit size of a **Tins** signature σ is

$$|\sigma| = 2\lambda + 64 + 2\lambda + (\lambda \cdot T_{open} + \tau \cdot 2\lambda) + \tau(2n - 4 + k).$$

Algorithm 16 $\text{Sign}(\mathbf{sk}, \text{msg})$

Require: : $\mathbf{sk} = (\text{seed}_{sk}, \text{seed}_{pk})$, $\text{msg} \in \{0, 1\}^*$

```

1:  $\text{salt} \leftarrow \text{TRG.Seed}(2\lambda)$ 
2:  $\text{rseed} \leftarrow \text{TRG.Seed}(\lambda)$ 
3:  $(\hat{\mathbf{u}}, \hat{\mathbf{v}}, \alpha, \beta) = \text{ExpandSecretVector}(\text{seed}_{sk}, \text{seed}_{pk})$ 
4:  $(\text{base}, \delta, h_{sh}, \text{key}, \text{aux}) = \text{CommitPoly}(\text{salt}, \text{rseed}, \alpha, \beta)$ 
5: for  $0 \leq e \leq \tau - 1$  do
6:    $(p_{mid}[e], p_{base}[e]) = \text{ComputePoly}(\text{base}[e], \delta[e], \alpha, \beta, \hat{\mathbf{u}}, \hat{\mathbf{v}})$ 
7: end for
8:  $h_{piop} = \text{Hash}_2(\text{seed}_{pk}, \text{salt}, \text{msg}, h_{sh}, p_{mid}[0], \dots, p_{mid}[\tau - 1], p_{base}[0], \dots, p_{base}[\tau - 1], v_{n-1})$ 
9:  $(\text{ctr}, \pi_{BAC}) = \text{OpenRandomEva}(\text{key}, h_{piop})$ 
10:  $\sigma = (\text{salt}, \text{ctr}, h_{piop}, \pi_{BAC}, p_{mid}, \text{aux})$ 
11: Output:  $\sigma$ 

```

4.4 Verification

In **Tins** signature verification $\text{Verify}(\mathbf{pk}, \sigma, \text{msg})$, verifier recomputes random challenge values using signature data, re-evaluates polynomial commitments at challenge points via public key parameters, recalculates hash digest h'_{piop} from recomputed values. Verify signature validity by comparing computed hash digest h'_{piop} with signature-contained hash digest h_{piop} .

Algorithm 17 *Verify*($\mathbf{pk}, \sigma, \text{msg}$)

Require: : $\mathbf{pk} = (\text{seed}_{pk}, v_{n-1})$, $\sigma = (\text{salt}, \text{ctr}, h_{piop}, \pi_{BAVC}, p_{mid}, aux)$, $\text{msg} \in \{0, 1\}^*$

- 1: $(\hat{\mathbf{u}}, v, v_{n-2}) = \mathbf{ExpandPublicVector}(\text{seed}_{pk})$
 - 2: $(\text{eval}, \text{point}, h_{sh}, v_{grinding}) = \mathbf{ComputeEva}(\text{salt}, \text{ctr}, h_{piop}, \pi_{BAVC}, aux)$
 - 3: **for** $0 \leq e \leq \tau - 1$ **do**
 - 4: $p_{base}[e] = \mathbf{RecomputePolyProof}(\text{point}, \text{eval}, \hat{\mathbf{u}}, \hat{\mathbf{v}}, p_{mid})$
 - 5: **end for**
 - 6: $h'_{piop} = \text{Hash}_2(\text{seed}_{pk}, v_{n-1}, \text{salt}, \text{msg}, h_{sh}, p_{mid}[0], p_{base}[0], \dots, p_{mid}[\tau - 1], p_{base}[\tau - 1])$
 - 7: **Output:** $h'_{piop} \stackrel{?}{=} h_{piop}$
-

5 Parameter Sets

According to the analysis in [11, 12], the NSBC problem requires selecting $n \approx k/2$ to ensure the hardness of the problem. To ensure **Tins** achieves λ security strength, we also require that the bitsize of the elements of $\mathbb{F}_{q^k}$ to be 2λ . We use a proof-of-work to the Fiat-Shamir hash computation of the last challenge, the parameters N, τ and ω will be chosen such that $(\frac{2}{N})^\tau \leq \frac{1}{2^{\lambda-\omega}}$ to achieve a λ -bit security. We choose the parameter T_{open} such that the probability that the number of revealed nodes is less than T_{open} is approximately 0.2. Based on the analyses in [11, 12] and [6], the parameters used in **Tins** are given in the following Table[2].

Table 2: Tins parameters

Instancel	q	k	n	μ	N	τ	T_{open}	ω	Classical bit security
Tins-128	2	276	140	12	2^{12}	12	125	6	137
Tins-256	2	528	268	12	2^{12}	24	255	4	261
Tins-512	2	1044	524	12	2^{12}	47	510	6	521

6 Security and Known Attacks

6.1 Security Proof

The **Tins** scheme is based on the NSBC problem, and its security relies on the following hardness assumption.

Assumption 1. *Given $\hat{\mathbf{u}} = (\mathbf{u}, u_{n-2}, u_{n-1}) \in \mathbb{F}_{q^k}^n$, $\hat{\mathbf{v}} = (\mathbf{v}, v_{n-2}, v_{n-1}) \in \mathbb{F}_{q^k}^n$, which are linearly independent over $\mathbb{F}_q$, it is hard to find two vectors $\boldsymbol{\alpha}, \boldsymbol{\beta} \in \mathbb{F}_q^{n-2}$ such that $(\langle \mathbf{u}, \boldsymbol{\alpha} \rangle + u_{n-2})(\langle \mathbf{v}, \boldsymbol{\beta} \rangle + v_{n-1}) = (\langle \mathbf{u}, \boldsymbol{\beta} \rangle + u_{n-1})(\langle \mathbf{v}, \boldsymbol{\alpha} \rangle + v_{n-2})$.*

In the proof by a sequence of games, we typically require the following lemma.

Lemma 1. *Let A, B , and F be three events in some probability space such that $\Pr[A \wedge \neg F] = \Pr[B \wedge \neg F]$. Then $|\Pr[A] - \Pr[B]| \leq \Pr[F]$.*

Based on the Lemma 1, the security result for the **Tins** scheme is as follows.

Theorem 1. *Let PRG be a (t, ϵ_{PRG}) -secure pseudorandom generator, let ϵ_{NSBC} be the advantage for an adversary to solve the NSBC problem, and let $\text{Hash}_1, \text{Hash}_2, \text{Hash}_3$ be modeled as random oracles. For any PPT adversary $\mathcal{A}$ attacking the Tins scheme, if it makes at most q_i queries to Hash_i (for $i \in [3]$) and q_s queries to the signing oracle $\mathcal{O}_{sig}$, then the advantage of $\mathcal{A}$ in succeeding is:*

$$\text{Adv}_{\mathcal{A}, \text{Tins}}^{\text{EUF-CMA}} \leq \frac{3 \cdot (q' + (\tau \cdot N + 1) \cdot q_s)^2}{2 \cdot 2^{2\lambda}} + \frac{q_s \cdot (q_s + 3q')}{2^{2\lambda}} + \tau \cdot q_s \cdot \epsilon_{PRG} + \epsilon_{NSBC} + \frac{1}{2^w} \cdot q' \cdot \left(\frac{2}{N}\right)^\tau$$

where $q' = \max(q_1, q_2, q_3)$ and τ is the number of repetitions of the signature.

Proof. We employ a proof by a sequence of experiments. Specifically, we define a sequence of five games G_i for $i \in [5]$, where G_1 is the real EUF-CMA game in which the challenger $\mathcal{C}$ runs the actual signature scheme. Let $Pr[Suc_i]$ be the probability that adversary $\mathcal{A}$ succeeds in game G_i . We therefore have:

$$Adv_{\mathcal{A}, Tins}^{EUF-CMA} = Pr[Suc_1].$$

Since

$$|Pr[Suc_1] - Pr[Suc_5]| \leq \sum_{i=1}^4 |Pr[Suc_i] - Pr[Suc_{i+1}]|,$$

that is

$$Pr[Suc_1] \leq \sum_{i=1}^4 |Pr[Suc_i] - Pr[Suc_{i+1}]| + Pr[Suc_5],$$

we only need to prove

$$|Pr[Suc_i] - Pr[Suc_{i+1}]| \leq \text{negl}(\cdot), i \in [4] \wedge Pr[Suc_5] \leq \text{negl}(\cdot).$$

Thus, the advantage of adversary $\mathcal{A}$ against the **Tins** scheme, $Adv_{\mathcal{A}, Tins}^{EUF-CMA}$, is negligible, proving the security of the Tins scheme.

–**Game G_1 :** This is the original EUF-CMA experiment. Specifically, the challenger $\mathcal{C}$ first runs the **KeyGen**() algorithm, sends the public key $\mathbf{pk} := (\hat{u}, \hat{v})$ to the adversary $\mathcal{A}$, and keeps the private key $\mathbf{sk} := (\alpha, \beta)$ for itself. Then, $\mathcal{A}$ can make at most q_i queries to each random oracle $Hash_i$ (for $i \in [3]$) and q_s queries to the signing oracle $\mathcal{O}_{sig}$. Finally, $\mathcal{A}$ outputs a signature pair (m, σ) . This pair is considered a forgery if it verifies successfully and the message m has not been previously queried to $\mathcal{O}_{sig}$. For game G_1 , the following holds:

$$Adv_{\mathcal{A}, Tins}^{EUF-CMA} = Pr[Suc_1] \tag{1}$$

–**Game G_2 :** Define event F_1 : a collision occurs among the outputs of $Hash_1$, $Hash_2$, or $Hash_3$. This game is identical to G_1 , except that it aborts if event F_1 occurs. Let $Pr[F_1]$ be the probability of event F_1 . According to Lemma 1, we have:

$$|Pr[Suc_1] - Pr[Suc_2]| \leq Pr[F_1].$$

To analyze $Pr[F_1]$, we first calculate the number of times the hash functions $Hash_1$, $Hash_2$, and $Hash_3$ are invoked during the interaction between the adversary $\mathcal{A}$ and the challenger $\mathcal{C}$. Since each signing query invokes $Hash_1$ once, $Hash_2$ once, and $Hash_3$ $(\tau \cdot N + 1)$ times, the total number of invocations for any single hash function is bounded by $q' + (\tau \cdot N + 1) \cdot q_s$, where $q' = \max(q_1, q_2, q_3)$. Furthermore, since the output space of $Hash_1$, $Hash_2$, and $Hash_3$ is of size $2^{2\lambda}$, it follows that:

$$Pr[F_1] \leq 3 \cdot C_{q' + (\tau \cdot N + 1) \cdot q_s}^2 \cdot \frac{1}{2^{2\lambda}} \leq \frac{3 \cdot (q' + (\tau \cdot N + 1) \cdot q_s)^2}{2 \cdot 2^{2\lambda}}.$$

That is

$$|Pr[Suc_1] - Pr[Suc_2]| \leq \frac{3 \cdot (q' + (\tau \cdot N + 1) \cdot q_s)^2}{2 \cdot 2^{2\lambda}}. \tag{2}$$

–**Game G_3 :** Define event F_2 : an input to $Hash_1$, $Hash_2$, or $Hash_3$ has appeared in a previous query. This game is identical to G_2 , except that it aborts if event F_2 occurs. Let $Pr[F_2]$ be the probability of event F_2 . According to the probability difference lemma, we have:

$$|Pr[Suc_2] - Pr[Suc_3]| \leq Pr[F_2].$$

Since the case of collisions among the outputs of $Hash_1$, $Hash_2$, and $Hash_3$ has been ruled out in this experiment, the occurrence of F_2 implies that at least one salt value is reused. Each query to the signing

oracle $\mathcal{O}_{sig}$ samples a salt, which can collide with a salt from a previous signature query or a salt from a previous $Hash_i$ ($i \in [3]$) query. It follows that:

$$Pr[F_2] \leq (C_{q_s}^1 \cdot C_{q_s+q_1+q_2+q_3}^1) \cdot \frac{1}{2^{2\lambda}} \leq \frac{q_s \cdot (q_s + 3 \cdot q')}{2^{2\lambda}}.$$

Then we have:

$$|Pr[Suc_2] - Pr[Suc_3]| \leq \frac{q_s \cdot (q_s + 3 \cdot q')}{2^{2\lambda}}. \quad (3)$$

–**Game G_4 :** This game is identical to G_3 , except that the outputs of $Hash_1$, $Hash_2$, and $Hash_3$ (namely, $h_{sh}, h_{piop}, commit_{e,i}, h_{com}$) are replaced with uniformly random values. The distinction after this replacement is that in G_3 , $Hash_1$, $Hash_2$, and $Hash_3$ act as random oracles, providing consistent outputs for identical queries. In this new experiment (G_4), however, each query is answered with a fresh, uniformly random value. In other words, G_4 and G_3 only differ if a query to $Hash_1$, $Hash_2$, or $Hash_3$ is repeated. However, this event is already handled as a halting condition in G_3 , meaning the experiment aborts before such a repetition can cause a divergence. Therefore, this situation never arises. Hence:

$$Pr[Suc_3] = Pr[Suc_4]. \quad (4)$$

–**Game G_5 :** This game is identical to G_4 , except that the views of each party are generated randomly by the simulator $\mathcal{S}$ from the zero-knowledge proof, instead of by a pseudo-random generator (PRG). Therefore, an adversary's ability to distinguish between G_4 and G_5 can be reduced to distinguishing between PRG values and truly random values. Thus, we have:

$$|Pr[Suc_4] - Pr[Suc_5]| \leq \tau \cdot q_s \cdot \epsilon_{PRG}. \quad (5)$$

where, ϵ_{PRG} is the adversary's advantage in distinguishing a PRG value from a truly random value.

At this point, it remains to analyze the probability that adversary $\mathcal{A}$ succeeds in G_5 , $Pr[Suc_5]$. Let F_3 be the event: the forgery output by adversary $\mathcal{A}$ can be used to solve the NSBC problem. Thus, we have:

$$Pr[Suc_5] = Pr[(Suc_5 \cap F_3) \cup (Suc_5 \cap \overline{F_3})] \leq Pr[Suc_5 \cap F_3] + Pr[Suc_5 \cap \overline{F_3}].$$

The event $Suc_5 \cap F_3$ means that adversary $\mathcal{A}$ successfully produces a forgery in experiment G_5 , and that this forgery can be used to solve the NSBC problem. This implies that the adversary must have used a genuine witness to generate the forgery. Since in G_5 , the outputs of all $Hash_i$ (for $i \in [3]$) are uniformly random values and the parties' views are also simulated, it must be possible to extract a solution to the NSBC problem from such a forgery; that is, $Pr[Suc_5 \cap F_3] > 0$. However, since the NSBC problem is assumed to be hard, it follows that:

$$Pr[(Suc_5 \cap F_3)] \leq \epsilon_{NSBC}.$$

Finally, we need to analyze an upper bound on $Pr[(Suc_5 \cap \overline{F_3})]$. The non-occurrence of F_3 means that adversary $\mathcal{A}$ does not possess a genuine witness (α, β) . To produce a corresponding forgery after querying $Hash_2$, the only way for $\mathcal{A}$ to succeed is to guess the value of $\{i^{*(e)}\}$:

- If the equality $(\langle \mathbf{u}, \alpha \rangle + u_{n-2})(\langle \mathbf{v}, \beta \rangle + v_{n-1}) = (\langle \mathbf{u}, \beta \rangle + u_{n-1})(\langle \mathbf{v}, \alpha \rangle + v_{n-2})$ does not hold, then since the polynomial $P(X)$ has at most 2 roots, the probability that the adversary successfully deceives $P(X)$ in a single round is

$$Pr[Suc_{round}] = \frac{2}{N}$$

Since generating a forgery requires repeating the above operation τ times, and the adversary $\mathcal{A}$ can query $Hash_2$ at most q' times, the upper bound on the probability of a successful forgery is $q' \cdot \left(\frac{2}{N}\right)^\tau$. Furthermore, since the ω -bit value $v_{grinding} \in \{0, 1\}^\omega$ must be zero, the adversary $\mathcal{A}$ has a success probability of $1/2^\omega$ for each iteration (e^*, ctr). Thus, the upper bound on the probability of a successful forgery by adversary $\mathcal{A}$ is

$$\frac{1}{2^\omega} \cdot q' \cdot \left(\frac{2}{N}\right)^\tau.$$

Finally, the adversary must ensure that $\pi_{BAVC} \neq \perp$, meaning the sibling path has not exceeded the threshold T_{open} . In other words, for the adversary to succeed in the challenge, the challenge value $\{i^{*(e)}\}$ must both fool the polynomial $P(X)$ and satisfy the condition $\pi_{BAVC} \neq \perp$. Therefore, $Pr[Suc_{round}]$ requires further analysis. Assuming $Pr[\pi_{BAVC} = \perp] = \theta$, the space of valid challenges is reduced from N^τ to $(N^\tau) \cdot (1 - \theta)$. We obtain:

$$Pr[Suc_{round}] = \frac{2^\tau}{(N^\tau) \cdot (1 - \theta)} \cdot (1 - \theta) = \left(\frac{2}{N}\right)^\tau.$$

Finally,

$$Pr[(Suc_5 \cap \overline{F_3})] = \frac{1}{2^\omega} \cdot q' \cdot \left(\frac{2}{N}\right)^\tau.$$

Hence,

$$Pr[Suc_5] \leq \epsilon_{NSBC} + \frac{1}{2^\omega} \cdot q' \cdot \left(\frac{2}{N}\right)^\tau. \quad (6)$$

Theorem 1 follows from Equations (1)-(6). $\square$

6.2 Known Attacks

Gröbner basis attack. The NSBC problem can be formulated as a system of bilinear equations. We can solve this bilinear system using Gröbner basis algorithms, for example the so-called F_5 algorithm. When $n \approx k/2$, we have the following result from [11].

Theorem 2. *The complexity of computing a Gröbner basis of the NBSC with the F_5 algorithm is upper bounded by*

$$O\left(\left(\binom{3k/2}{k/2}\right)^\omega\right),$$

where $2 \leq \omega \leq 3$ is the linear algebra constant.

Generic collision attack. When $n \approx k/2$, Huth[11] describes a highly effective attack against the SBC problem, with a complexity of approximately $O(q^{k+1-n})$.

LRC attack. Recently, Briaud [6] observed that an instance of the SBC problem is equivalent to an instance of the Low Rank Codeword (LRC) problem. Consequently, they used an LRC solver to derive an attack of the SBC problem, with a complexity of approximately

$$O(q^{n-2} T_{LRC}(q, m, n - h, l, r - h)),$$

where $T_{LRC}(q, m, n - h, l, r - h)$ is the time complexity of an arbitrary LRC solver.

Based on the aforementioned attack method and complexity estimation, we can obtain the attack complexity for a specific instance of the Tins signature, which are given in the following Table.

Table 3: Complexity of the known attacks

Instance	Gröbner basis attack	Generic collision attack	LRC attack
Tins-128	2^{380}	2^{137}	2^{138}
Tins-256	2^{728}	2^{261}	2^{266}
Tins-512	2^{1440}	2^{521}	2^{522}

7 Performance

7.1 Implementation

We implemented the algorithm in C. The benchmark environment for the code is as follows:

Host machine: Windows 10, 12th Gen Intel Core i5-1240P (64-bit PC processor, base frequency 1.70 GHz), 16 GB DDR4 memory.

Virtualization: WSL2.

Virtual machine allocation: 8 vCPUs (as shown by nproc), 8 GiB memory allocated (total memory shown by free -h is 7.6 GiB, Swap 2.0 GiB).

Guest operating system: Ubuntu 20.04.2 LTS.

Compiler: GCC 9.4.0.

Table 4 presents the efficiency of the algorithm under three security strength levels. The efficiency values are averages over 50 random seeds and messages. Notably, the primary computational overhead for both signature generation and verification originates from the extensive invocation of hash functions. We analyzed the peak physical memory usage of the reference implementation code throughout its entire execution lifecycle; the memory consumption of Tins-128, Tins-256, and Tins-512 was 14.44MB, 44.61MB, and 134.74MB, respectively.

Table 4: Performances of **Tins** in Millions of CPU Cycles(reference implementations)

Instance	Keygen	Sign	Verify
Tins-128	0.757 M	466.792 M	437.606 M
Tins-256	4.722 M	1780.665 M	1727.229 M
Tins-512	35.794 M	8440.860 M	8392.468 M

Public key, secret key and signatures size of **Tins** are given in the following Table[5].

Table 5: Keys and signature sizes of Tins

Instancel	<i>sk</i> size[B]	<i>pk</i> size[B]	σ size[B]
Tins-128	32	51	3284
Tins-256	64	98	13012
Tins-512	128	195	51187

Table 6 shows the efficiency of the algorithm after optimization. We adopted the AVX instruction set for some optimizations, but the computational efficiency did not improve significantly. This is because the main time consumption of the **Tins** algorithm lies in the massive calls to low-level hash functions. We needed to keep the hash functions and related auxiliary functions completely unchanged during optimization. If we optimize the underlying hash functions, the efficiency of the algorithm will be greatly improved. The efficiency values are averages over 50 random seeds and messages.

Table 6: Performances of **Tins** in Millions of CPU Cycles(optimized implementations)

Instance	Keygen	Sign	Verify
Tins-128	0.396 M	460.968 M	433.176 M
Tins-256	2.037 M	1732.546 M	1668.100 M
Tins-512	6.564 M	8342.944 M	8233.093 M

Table 7 presents the efficiency of the additional implementation of the algorithm. The **Tins** algorithm inherently has the advantage of being multi-core parallelizable; therefore, we accelerated it using multi-core parallel computing without modifying the underlying hash functions and related auxiliary functions.

Table 7: Performances of **Tins** in Millions of CPU Cycles(additional implementations)

Instance	Keygen	Sign	Verify
Tins-128	0.420 M	195.049 M	140.418 M
Tins-256	2.124 M	732.848 M	613.132 M
Tins-512	8.595 M	2574.758 M	2542.490 M

7.2 Known Answer Test Values

Known Answer Test (KAT) values have been generated using the reference implementations and the script provided by the ICCS, which can be retrieved in the Test_Vector/ folder.

8 Advantages and Limitations

8.1 Advantages

Simplicity. The **Tins** algorithm is concise and easy to understand. Its underlying hard problem is simpler and easier to describe than those of similar signature frameworks such as Mirath and SDitH. Furthermore, in terms of the signature design framework, compared to the HJ2025[12] scheme based on the same hard problem, the **Tins** algorithm features a more concise framework and computational process, facilitating development and implementation.

Signature+public key size. **Tins** offers highly competitive signature sizes. For instance, at the 128-bit security level, the signature size of **Tins** is smaller than those of similar “in-the-head” frameworks such as SDitH and FAEST. Furthermore, benefiting from a smaller public key size, at the 128-bit security level, the combined size of the signature and public key is smaller than that of the NIST-standardized algorithms ML-DSA (Dilithium) and SLH-DSA (SPHINCS+). At the 128-bit security level, the combined signature and public key size for **Tins** is 3.28 kB, whereas ML-DSA (Dilithium) requires 3.73 kB and SLH-DSA (SPHINCS+) requires 7.89 kB.

8.2 Limitations

Signature Size Growth Rate. To increase the security level, the instances of the hard problem underlying **Tins** need to increase significantly in size, and the signature requires many internal repetition rounds. This leads to a substantial growth in the signature size of **Tins** as the security level increases. For example, at the 256-bit security level, although the combined signature and public key size of **Tins** is smaller than that of SLH-DSA (SPHINCS+), it is larger than that of ML-DSA (Dilithium).

Lightweight Devices. **Tins** signatures may incur significant overhead for lightweight IoT devices, such as low-cost and embedded devices.

References

- [1] Gora Adj, Nicolas Aragon, Stefano Barbero, and et.al. Mirath signature scheme. *NIST Post-Quantum Cryptography: Additional Digital Signature Schemes. Round 2 Additional Signatures*, 2025.

- [2] Carlos Aguilar, Melchor Slim, Betttaieb, Loic Bidoux, and et.al. The syndrome decoding in the head (sd-in-the-head) signature scheme. *NIST Post-Quantum Cryptography: Additional Digital Signature Schemes. Round 2 Additional Signatures*, 2025.
- [3] Nicolas Aragon, Magali Bardet, Loic Bidoux, and et.al. Ryde signature scheme. *NIST Post-Quantum Cryptography: Additional Digital Signature Schemes. Round 2 Additional Signatures*, 2025.
- [4] Carsten Baum, Ward Beullens, Lennart Braun, and et.al. Faest v2. *NIST Post-Quantum Cryptography: Additional Digital Signature Schemes. Round 2 Additional Signatures*, 2025.
- [5] Carsten Baum, Ward Beullens, Shibam Mukherjee, Emmanuela Orsini, Sebastian Ramacher, Christian Rechberger, Lawrence Roy, and Peter Scholl. One tree to rule them all: Optimizing ggm trees and owfs for post-quantum signatures. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 463–493. Springer, 2024.
- [6] Pierre Briaud and Romaric Neveu. Cryptanalysis of the subfield bilinear collision problem. *Cryptology ePrint Archive*, 2026.
- [7] Thibauld Feneuil. The polynomial-iop vision of the latest mpcith frameworks for signature schemes. In *Post-Quantum Algebraic Cryptography-Workshop*, volume 2, 2024.
- [8] Thibauld Feneuil and Matthieu Rivain. Threshold linear secret sharing to the rescue of mpc-in-the-head. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 441–473. Springer, 2023.
- [9] Thibauld Feneuil and Matthieu Rivain. Threshold computation in the head: Improved framework for post-quantum signatures and zero-knowledge arguments. *Journal of Cryptology*, 38(3):28, 2025.
- [10] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Conference on the theory and application of cryptographic techniques*, pages 186–194. Springer, 1986.
- [11] Janik Huth and Antoine Joux. MPC in the head using the subfield bilinear collision problem. In *Annual International Cryptology Conference*, pages 39–70. Springer, 2024.
- [12] Janik Huth and Antoine Joux. VOLE-in-the-head signatures from subfield bilinear collisions. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 3–34. Springer, 2025.
- [13] Yuval Ishai, Eyal Kushilevitz, Rafail Ostrovsky, and Amit Sahai. Zero-knowledge from secure multiparty computation. In *Johnson, D.S., Feige, U. (eds.) 39th ACM STOC*, pages 21–30. ACM Press, 2007.